

System Architecture and Implementation

The system is built around a core data processing pipeline that transforms the raw CACM collection into a structured and searchable inverted index. This process fulfills the primary requirement of using an inverted index for retrieval.

1. Document Parsing

The system first parses the `cacm.all` file to extract document content. It identifies document boundaries (marked by `.I`) and extracts the text from the title (`.T`) and abstract (`.W`) fields. This content is then associated with its unique document ID.

2. Text Preprocessing

To prepare the text for indexing, a series of preprocessing steps are applied:

- **Tokenization:** The raw text is converted to lowercase, and all non-alphabetic characters are removed. The cleaned text is then split into a list of individual words or tokens.
- **Stopword Removal:** To comply with the requirement of using a stop list, the system dynamically identifies the seven most frequent terms across the entire collection. These common words, which typically offer little semantic value, are removed from each document's token list.
- **Stemming:** The Porter Stemmer algorithm is used to reduce the remaining tokens to their base or root form (e.g., "retrieval", "retrieving" -> "retriev"). This helps group related words under a single term, improving query matching.

3. Inverted Index Construction

An inverted index is created from the preprocessed tokens. The index is a dictionary where each key is a unique stemmed term (from the vocabulary) and its value is a postings list. Each postings list contains tuples of `(document_ID, term_frequency)`, allowing for rapid lookup of all documents that contain a given term.

Retrieval and Testing

The system provides functionality for single-word queries and was tested using three distinct terms.

1. Query Processing and Retrieval

When a single-term query is submitted, it undergoes the same preprocessing steps as the document text: it is lowercased and stemmed. The system then looks up the processed term in the inverted index to find its postings list. The documents in the postings list are ranked in descending order based on the term's frequency (TF) within each document.

2. Test Data

As required, the system was tested with three different queries: "algorithm", "problem", and "computer". For each query, the system retrieves and ranks all matching documents. To facilitate manual review and relevance assessment, the top 25 retrieved documents for each test query were saved to a corresponding output file. This demonstrates the system's retrieval capabilities on concrete examples.

